Reviewer Pack — Khinchin’s Constant Bounds Boolean Circuit Entropy

Entry 5d03b372048e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 03:26:31 UTC

Khinchin’s Constant Bounds Boolean Circuit Entropy

Entry ID: `5d03b372048e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 03:26:31 UTC

1. Conjecture

Field A (mathematical branch): Statistical Mechanics (specifically, ergodic theory) **Field B** (complexity object): Boolean circuit complexity

Statement:

> For any given boolean circuit C with n inputs and output size m, the entropy of C, denoted as H(C), satisfies |H(C) - 2/H(π_0)| ≤ 1/π_0, where π_0 is Khinchin’s constant.

Rationale (proposer’s reasoning):

> Khinchin’s constant arises in the context of dynamical systems and has been used to study the entropy of measure-preserving transformations. Applying this concept to boolean circuits could reveal a connection between the statistical properties of circuits and their computational complexity, potentially leading to new insights into circuit lower bounds.

Taxonomy category: `STATISTICAL_MECHANICS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `57a9cc485d99c7c4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a boolean circuit C with n inputs and output size m, |H(C) - 2/H(π_0)| ≤ 1/π_0 where π_0 is Khinchin’s constant.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Khinchin's constant" AND "Boolean circuit entropy"` - `"ergodic theory" AND "Boolean circuit complexity"` - `"statistical mechanics" AND "circuit complexity"`

Top relevant hits considered: - [s2:10.1145/28395.28404] Algebraic methods in the theory of lower bounds for Boolean circuit complexity - [s2:2511.21738] On the Incompressibility of Truth With Application to Circuit Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Khinchin's constant
    pi_0 = 2.718281828459045

    # Generate a random boolean circuit with n inputs and output size m
    n = random.randint(5, 40)
    m = random.randint(1, 2)

    # Calculate the entropy H(C) of the circuit using Shannon's formula
    # For simplicity, we assume each gate has an equal probability of being true or false
    # This is a very simplified model and not representative of real circuits
    entropy_C = -m * (math.log(m / 2**n, 2) + (1 - m) * math.log((1 - m) / (2**(n-1)), 2))

    # Calculate the difference |H(C) - 2/H(π_0)|
    diff = abs(entropy_C - 2 / pi_0)

    return {
        "metric_name": "entropy_difference",
        "metric_value": diff,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": diff <= 1 / pi_0,
        "counterexample": "" if diff <= 1 / pi_0 else f"Entropy difference {diff} exceeds threshol
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67,
        71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]
```

```
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    # Compute mean/std of metric_value and fraction of seeds where conjecture_holds
    if all("metric_value" in r for r in results):
        mean = sum(r["metric_value"] for r in results) / len(results)
        std = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if all(r["conjecture_holds"] for r in results):
            print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
        elif support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
        else:
            first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), N
            print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_fail
    else:
        print("RESULT: INCONCLUSIVE missing_metric_value")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7adf9e5f.py", line 53, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7adf9e5f.py", line 31, in run_trial
    entropy_C = -m * (math.log(m / 2**n, 2) + (1 - m) * math.log((1 - m) / (2**(n-1)), 2))
                                                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: math domain error
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Review and debug the test code to ensure it can run successfully and produce the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	25819
2	preregistration	ollama_remote	glm4:latest	0	0	16970
3	novelty	ollama_remote	glm4:latest	0	0	11364
4	novelty	ollama_remote	glm4:latest	0	0	16597
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16484
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10767
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6071
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17851
9	judge	ollama_remote	glm4:latest	0	0	12047

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 133971 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5d03b372048e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5d03b372048e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5d03b372048e.tar.gz` (if generated)